Sovereign AI

Fine-Tuning Mistral 7B on Google Colab — Phase-Wise Build Guide

QLoRA Fine-Tuning | BFSI Domain Adaptation | Hugging Face + Ollama Deployment

---

A practical, code-level companion to the 12-Phase Sovereign AI BFSI LLM Roadmap

Prepared for: Sovereign AI project
Date: 23 September 2026

Colab Reality Check

This guide starts from Mistral 7B's existing pretrained weights (Track A from the main roadmap) rather than training from scratch. Mistral 7B's native tokenizer already has a 32,000-token vocabulary, so BFSI terms are added to it rather than building a separate 32K tokenizer from scratch. Full pretraining and full-parameter fine-tuning are not realistic on Colab (100+GB VRAM needed) — everything here uses QLoRA (4-bit quantized base + trainable LoRA adapters), the standard way to fine-tune a 7B model on a single Colab GPU.

Tier	GPU	VRAM	Session limit	Fits Mistral 7B fine-tuning?
Free	T4	16GB	~12h, frequent disconnects	Yes — 4-bit QLoRA + short sequences only
Colab Pro	A100 / V100	40GB	Longer, more stable	Yes — comfortably, QLoRA or 8-bit
Colab Pro+	A100	40GB	Background execution	Best option for multi-hour runs

1 Environment Setup

Set the Colab runtime to GPU (Runtime > Change runtime type > GPU: T4 or A100), install libraries, authenticate with Hugging Face, and mount Google Drive so data/checkpoints survive a disconnect.

```
!pip install -q -U transformers accelerate peft bitsandbytes trl datasets sentencepiece
huggingface_hub

from huggingface_hub import login
login() # paste an HF token with read access

from google.colab import drive
drive.mount('/content/drive')
WORKDIR = "/content/drive/MyDrive/sovereign-ai-bfsi"
import os; os.makedirs(WORKDIR, exist_ok=True)
```

2 Data Collection

At Colab scale, aim for 500–5,000 instruction examples to start — not billions of tokens (that is pretraining/DAPT scale, out of reach here). Upload as JSONL to Drive: BFSI Q&A pairs, summarization examples, regulatory-clause explanations, classification tasks.

```
{"instruction": "Explain what NPA means for a bank's balance sheet.", "response": "..."}
{"instruction": "Summarize the KYC requirements in this clause: ...", "response": "..."}
```

If you don't yet have proprietary BFSI data, seed with a public Hugging Face finance dataset to validate the pipeline first, then swap in your real corpus:

```
from datasets import load_dataset
ds = load_dataset("json", data_files=f"{WORKDIR}/bfsi_train.jsonl")["train"]
```

3 Data Cleaning & Formatting

Format every example into Mistral's chat template — this matters more than heavy cleaning at this scale.

```
def to_mistral_format(ex):
return {"text": f"<s>[INST] {ex['instruction']} [/INST] {ex['response']}</s>"}

ds = ds.map(to_mistral_format)
ds = ds.filter(lambda ex: len(ex["text"]) > 20) # drop empties
ds = ds.train_test_split(test_size=0.1, seed=42)
```

For real BFSI data, also run: dedup (drop exact/near-duplicate rows), PII masking (account numbers/PAN via regex), and length filtering before this step.

4 Tokenizer Extension — the 32K BFSI Vocab

Mistral's tokenizer is already 32K — extend it rather than rebuilding one from scratch.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-v0.1")
print(tokenizer.vocab_size) # 32000

# Discover BFSI terms that fragment badly today
bfsi_terms = ["NPA", "CASA", "IRDAI", "AML", "KYC", "moratorium", "IFSC", "ISIN", "NBFC",
"SLR", "CRR"]
for t in bfsi_terms:
print(t, "->", tokenizer.tokenize(t))

new_tokens = [t for t in bfsi_terms if len(tokenizer.tokenize(t)) > 2]
tokenizer.add_tokens(new_tokens)
tokenizer.pad_token = tokenizer.eos_token
```

Skip this step for a first pass if you want to move fast — Mistral's existing BPE tokenizer already handles most BFSI English reasonably. Add tokens once you've measured real fragmentation on your own corpus.

5 Load Model in 4-bit (QLoRA)

```
import torch
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

bnb_config = BitsAndBytesConfig(
load_in_4bit=True,
bnb_4bit_quant_type="nf4",
bnb_4bit_compute_dtype=torch.bfloat16,
bnb_4bit_use_double_quant=True,
)

model = AutoModelForCausalLM.from_pretrained(
"mistralai/Mistral-7B-v0.1",
quantization_config=bnb_config,
device_map="auto",
)

if new_tokens: # only if you extended the tokenizer in Phase 4
model.resize_token_embeddings(len(tokenizer))

model.gradient_checkpointing_enable()
model.enable_input_require_grads()
```

6 LoRA Configuration

```
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

model = prepare_model_for_kbit_training(model)

lora_config = LoraConfig(
r=16,
lora_alpha=32,
target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
lora_dropout=0.05,
bias="none",
task_type="CAUSAL_LM",
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters() # expect ~0.1-0.3% of total params trainable
```

7 Fine-Tuning (SFT)

```
from trl import SFTTrainer, SFTConfig

sft_config = SFTConfig(
output_dir=f"{WORKDIR}/checkpoints",
per_device_train_batch_size=2, # T4: keep at 1-2; A100: can go to 4-8
gradient_accumulation_steps=8, # effective batch size 16
num_train_epochs=3,
learning_rate=2e-4,
bf16=True, # use fp16=True instead on T4 (no native bf16)
logging_steps=10,
save_strategy="epoch",
max_seq_length=1024, # reduce to 512 if you hit OOM on T4
report_to="none",
)

trainer = SFTTrainer(
model=model,
train_dataset=ds["train"],
eval_dataset=ds["test"],
args=sft_config,
dataset_text_field="text",
)

trainer.train()
trainer.save_model(f"{WORKDIR}/mistral-bfsi-adapter")
tokenizer.save_pretrained(f"{WORKDIR}/mistral-bfsi-adapter")
```

On a free T4, expect roughly 1–3 hours for a few thousand examples over 3 epochs. Checkpointing to Drive every epoch (set above) means a disconnect doesn't lose progress.

8 Evaluation

```
def ask(prompt, max_new_tokens=200):
inputs = tokenizer(f"<s>[INST] {prompt} [/INST]", return_tensors="pt").to(model.device)
out = model.generate(**inputs, max_new_tokens=max_new_tokens, do_sample=False)
return tokenizer.decode(out[0], skip_special_tokens=True)

test_prompts = [
"What does CRR stand for and how does it affect bank liquidity?",
"Summarize the key KYC red flags in a suspicious transaction pattern.",
]
for p in test_prompts:
print(p, "\n->", ask(p), "\n")
```

Build a small held-out BFSI test set (20–50 Q&A pairs not used in training) and score correctness manually or with an LLM-judge call — at this scale manual review is faster and more trustworthy than building automated eval infrastructure.

9 Merge Adapter & Save Full Model

For deployment (especially to Ollama), merge the LoRA weights into the base model so you have one standalone model.

```
# Reload base in fp16 (not 4-bit) to merge cleanly
base_model = AutoModelForCausalLM.from_pretrained(
"mistralai/Mistral-7B-v0.1", torch_dtype=torch.float16, device_map="auto"
)
if new_tokens:
base_model.resize_token_embeddings(len(tokenizer))

from peft import PeftModel
merged = PeftModel.from_pretrained(base_model, f"{WORKDIR}/mistral-bfsi-adapter")
merged = merged.merge_and_unload()

merged.save_pretrained(f"{WORKDIR}/mistral-bfsi-merged", safe_serialization=True)
tokenizer.save_pretrained(f"{WORKDIR}/mistral-bfsi-merged")

# Optional: push to Hugging Face Hub for safekeeping/sharing
merged.push_to_hub("your-username/mistral-7b-bfsi")
tokenizer.push_to_hub("your-username/mistral-7b-bfsi")
```

10 Convert to GGUF for Ollama

Colab can run this, though it's the slowest step (7B fp16 → GGUF conversion + quantization).

```
!git clone https://github.com/ggerganov/llama.cpp
!pip install -q -r llama.cpp/requirements.txt

!python llama.cpp/convert_hf_to_gguf.py {WORKDIR}/mistral-bfsi-merged \
--outfile {WORKDIR}/mistral-bfsi.gguf --outtype f16

# Build llama.cpp's quantize tool, then quantize down for practical serving size
!cmake -B llama.cpp/build llama.cpp && cmake --build llama.cpp/build --target llama-quantize
-j
!./llama.cpp/build/bin/llama-quantize {WORKDIR}/mistral-bfsi.gguf \
{WORKDIR}/mistral-bfsi-Q4_K_M.gguf Q4_K_M
```

Download mistral-bfsi-Q4_K_M.gguf from Drive to your local machine.

11 Deploy Locally with Ollama

On your own machine (not Colab), create a Modelfile:

```
FROM ./mistral-bfsi-Q4_K_M.gguf
TEMPLATE "[INST] {{ .Prompt }} [/INST]"
SYSTEM "You are a BFSI domain assistant specialized in banking, financial services, and
insurance."
PARAMETER temperature 0.3
```

Then build and run it:

```
ollama create mistral-bfsi -f Modelfile
ollama run mistral-bfsi "Explain the difference between NPA and NPL."
```

12 Monitoring & Iteration

At Colab scale, keep this lightweight: log real usage prompts/outputs to a file or sheet, review weekly, add corrected examples back into your Phase 2 JSONL, and re-run Phases 5–9 periodically. Each fine-tuning run is cheap (a few hours), so retraining monthly or quarterly as your corpus grows is realistic even on free-tier Colab.

Practical Tips for Colab

- Free tier disconnects mid-training are common — the save_strategy="epoch" checkpoint lets you resume with trainer.train(resume_from_checkpoint=True).
- If you hit CUDA out-of-memory on T4: drop max_seq_length to 512, per_device_train_batch_size to 1, and increase gradient_accumulation_steps.
- Keep raw data, checkpoints, and the merged model all under your mounted Drive folder — Colab's local disk is wiped on disconnect.
- Restart the runtime between the fine-tuning phase (Phases 5-8) and the merge/GGUF phase (Phases 9-10) if you hit memory pressure — loading the base model twice (4-bit, then fp16) in one session can exhaust VRAM.